Knowledge Center

OWASP Top 10 Web App Security Risks

What is OWASP Top 10?

The Open Web Application Security Project (OWASP) is a non-profit organization that provides guidance on how to develop and maintain secure software applications. OWASP is famous for its Top 10 list of web application security vulnerabilities, which lists the most important security risks affecting web applications.

The OWASP Top 10 list is based on community research and provides data on common vulnerabilities and exploits. It is revised every few years to reflect changes in the industry, such as how common certain attacks are, their business impact and the ease of exploitation.

Even more importantly, the OWASP Top 10 describes each category of application security risks, shows developers how to avoid them in the first place, and provides best practices for remediating them if they already exist.

The first version of the OWASP Top 10 List was released in 2003. Subsequent updates were made in 2004, 2007, 2010, 2013, 2017, and 2021.

The information below is based on the OWASP Top 10 list for 2021. Note that OWASP Top 10 security risks are listed in order of importance—so A1 is considered the most severe security issue, A2 is next, and A10 is the least severe of the top 10.

A1. Broken Access Control

When access control is breached, an attacker can gain access to user accounts, admin panels, databases, servers, sensitive information, business-critical applications, and other sensitive assets. It can allow unauthorized users to modify privileges to their advantage, and perform destructive operations such as tampering with data or destroying it.

OWASP recommends the following for mitigation:

- Least Privileges Approach

- Build strong access control with role-based authentication mechanisms
- Deny basic access to features except public resources
- Keep your servers lean by turning off unnecessary services and removing inactive and unnecessary accounts
- If you have multiple access points, disable unnecessary access points.
- Rate limiting API and controller access
- Sensitive data should not be stored in the root directory.
- The server directory listing must be disabled.

A2. Cryptographic Failures

Cryptographic failures (formerly listed in the Top 10 as “sensitive data exposure”) moved from position 3 to 2. It emphasizes encryption errors or lack of encryption that can lead to the exposure of sensitive data.

OWASP recommends the following for mitigation:

- Encrypt all data at rest using secure and trusted encryption algorithms, keys and protocols.
- Encrypt all data in transit using modern security protocols such as TLS.
- Identify and enforce strong security controls for all sensitive data.
- Do not collect and store sensitive data unless absolutely necessary.
- Do not cache sensitive data or data collection forms.
- Disable form autocomplete.
- Store passwords using strong, proven hash functions.

A3. Injections

Injection is an attack against a website that exploits vulnerabilities in the database or other part of the operating environment. Most injection attacks rely on a web application’s inability to distinguish user inputs from its own code. The attacker can then run malicious code in the application context, gaining access to protected areas and sensitive data.

Injection attacks might use structured query language (SQL) to retrieve information or perform a database operation that the attacker should not be allowed to perform. Other types of injection include command injection, which occurs at the operating system level, carriage return line feed (CRLF) injection, and lightweight directory access protocol (LDAP) injection.

OWASP recommends the following for mitigation:

- Adopt an API that completely bypasses the interpreter, use parameterized queries, or move to an object-relational mapping (ORM) approach.
- Use allowlist validation for inputs on the server side. Most injection attacks rely on special characters, and allowlists defining which characters or inputs are allowed can reduce the risk. However they are not foolproof.
- Use LIMIT and other SQL constraints in queries to avoid exposing large amounts of data in case of SQL injection.

A4. Insecure Design

This is a new category introduced by OWASP in 2021. It focuses on design and architectural flaws. Avoiding them requires careful threat modeling, taking security into consideration at the software design stage, and using reference architectures.

OWASP recommends the following for mitigation:

- Integrate security from the start of the software development lifecycle (SDLC).
- Build ready-to-use libraries of security-oriented design patterns, components, and frameworks for new applications and avoid building from scratch.
- Use threat modeling to design critical functions such as access control, authentication, business logic, and key flow.
- Add security concerns and controls into every user story developed as part of a software release.
- Divide the application into tiers and identify attack scenarios for each tier.
- Use plausibility testing to check whether certain inputs are acceptable at all, going from frontend to backend.

A5. Security Misconfigurations

Common setup issues, such as incorrect access control configuration, can allow attackers to quickly and easily gain access to sensitive data and application functions. These include inappropriate permissions, unnecessary feature activation, use of default accounts and passwords, misconfigured HTTP headers, and detailed error messages.

OWASP recommends the following for mitigation:

- Define a clear, easy deployment process that enforces application hardening.
- Use preconfigured templates, each with different credentials, to ensure identical configuration of development, testing, and production environments.
- Maintain a securely configured container image registry.

- Remove unused features and services and deploy applications with minimal configuration.
- Regularly update and patch applications.
- Use automated workflows to validate security configurations and detect misconfigurations, and fix any discovered issues immediately.

A6. Vulnerable and Outdated Components

Most web applications use third-party components, either open source or proprietary. These components contain code that is outside the organization's control, which can lead to undesirable outcomes like accent control violations and injection attacks.

A software component could be insecure, no longer supported by the software vendor, or in need of security updates. If the component contains vulnerabilities, this can compromise the entire application. Commonly used third-party components include application and web servers, operating systems, database management systems (DBMSs), APIs, open source libraries, and runtime environments.

OWASP recommends the following for mitigation:

- Maintain an up-to-date inventory of all components used by applications and their versions.
- Continuously scans components, libraries, and their dependencies for vulnerabilities.
- Keep all components up to date. Apply a virtual patch (a security policy or rule that can protect against exploit) if a patch is not immediately available from the vendor.
- Remove deprecated or unneeded components, features, and dependencies from your application.
- Only use components and third party software from official and trusted sources.

A7. Identification and Authentication Failures

Functions related to user authentication and session management, if not properly implemented, can expose users to security credentials, grant excessive privileges, or enable users to impersonate other identities.

OWASP recommends the following for mitigation:

- Enforce the use of multi-factor authentication.
- Do not use default credentials, especially administrator privileges.
- Implement a strong password policy.
- Deploy a secure session manager that generates timed session IDs.

- Monitor failed login attempts and set limits and delays.
- Use strong user registration and credential recovery processes.

A8. Software and Data Integrity Failures

Data integrity is becoming a primary concern for software security. This is a new category introduced by OWASP in 2021, which focuses on the integrity of software updates, critical application data, and CI/CD pipelines. A software and data integrity failure occurs when any of these are tampered with by an attacker, and other components within the application do not verify their integrity.

OWASP recommends the following for mitigation:

- Use digital signatures or similar mechanisms to verify that data or software has not been tampered with and that it came from its intended source.
- Use software supply chain security tools such as OWASP CycloneDX and OWASP Dependency-Check to ensure that components are free of design flaws.
- Ensure that the CI/CD pipeline uses segmentation, access control, and parameterization to protect code integrity from build through to production deployment.
- Do not send unsigned or unencrypted compiled data to untrusted clients, unless measures have been taken to identify tampering or duplication of the data.

A9. Security Logging and Monitoring Failures

When suspicious behavior occurs in an application and logging and monitoring are not in place, security breaches are much more likely to be successful. This category focuses on identifying, escalating, and resolving security incidents. Detecting a breach is almost impossible without logging and monitoring.

OWASP recommends the following for mitigation:

- Instantly detect suspicious activity with out-of-the-box logging and auditing software.
- Make sure logs are contextual and available in a format that enables in-depth forensic analysis.
- Implement security controls to prevent attackers from tampering with log data.

A10. Server-side Request Forgery (SSRF)

This category was added to the OWASP Top 10 list in 2021 because it was the top vulnerability voted in the OWASP Top 10 Community Survey. An SSRF vulnerability allows an attacker to access data on a remote resource based on an unauthenticated, custom URL.

Even servers protected by a firewall or VPN can be vulnerable to this vulnerability, if they accept unvalidated user input.

OWASP recommends the following for mitigation:

- Always perform validation of user input and sanitize all inputs.
- If an application has remote resource access functionality, ensure it is isolated from other aspects of the application.
- Block unsolicited incoming traffic with default deny firewall policy.
- Prevent clients from receiving raw responses.
- Create an allowlist of ports, destinations, and URL schemes.
- Disable HTTP Redirection.

Application Security with HackerOne

HackerOne and the community of ethical hackers is at the forefront of using OWASP to strengten application security and make the Internet safer by referencing the OWASP Top 10 to prioritize their actions. Taking this approach one step further, the HackerOne Global Top 10 can enable application security teams to increase their effectiveness with timely insights, segmented by industry and fueled by exploitable findings submitted by ethical hackers. These findings are often new or found by innovative techniquies and are unlikely to show up in the OWASP database. Combined, OWASP and HackerOne exploit databases assure that high severety vulnerablities are found and fixed before bad actors can do their work.

Learn more about the HackerOne approach to Application Security.

Contacted by a hacker?

Cybersecurity

Attacks

DevSecOps

Cookie Preferences Policies Terms Privacy Security ©2025 HackerOne All rights reserved.

Trust